

Udhëzues Informativ mbi Express.js

1. Çfarë është Express.js?

Express.js (ose thjesht Express) është një kornizë pune (framework) e lehtë, fleksibël dhe minimaliste për Node.js. Ajo ofron një grup të fuqishëm veçorish për zhvillimin e aplikacioneve ueb dhe API-ve RESTful, duke thjeshtuar menaxhimin e rrugëtimit (routing), kërkesave HTTP dhe përgjigjeve të serverit.

2. Origjina dhe Rëndësia

Express u krijua nga TJ Holowaychuk në vitin 2010 dhe frymëzohej nga korniza minimalistë Sinatra e gjuhës Ruby. Më vonë, projekti u bë pjesë e OpenJS Foundation. Express është korniza më popullore në ekosistemin e Node.js dhe përbën bërthamën e stiveve të njohura të zhvillimit si MERN (MongoDB, Express, React, Node) dhe MEAN (MongoDB, Express, Angular, Node).

3. Karakteristikat Kryesore

- Minimaliste dhe unopinionated:** Nuk imponon një strukturë fikse skedarësh apo bazë të dhënash, duke i dhënë zhvilluesit liri të plotë arkitekturore.
- Sistem Rrugëtimi i Fuqishëm (Routing):** Ofron mënyra të lehta për të përcaktuar se si aplikacioni i përgjigjet kërkesave në URL të caktuara bazuar në metodat HTTP (GET, POST, PUT, DELETE).
- Përdorim i Middleware:** Arkitektura e saj bazohet në funksione middleware për përpunimin e kërkesave, verifikimin e autentikimit, menaxhimin e gabimeve dhe më shumë.
- Integrim i Lehtë me Motorët e Temave:** Mbështet lehtësisht rendering-un në anën e serverit përmes motorëve si EJS, Pug apo Handlebars.

4. Shembull i një API Bazë në Express

Kodi i mëposhtëm tregon krijimin e një serveri të thjeshtë dhe një endpoint-i REST:

```
const express = require('express'); const app = express(); const port = 3000; // Middleware për të lexuar trupin e kërkesave JSON app.use(express.json()); app.get('/api/shembull', (req, res) => { res.json({ mesazhi: 'Përshëndetje nga Express.js!', statusi: 'sukses' }); }); app.listen(port, () => { console.log(`Serveri Express po punon në portën ${port}`); });
```

5. Koncepti i Middleware

Në Express.js, një middleware është një funksion që ka qasje në objektin e kërkesës (req), objektin e përgjigjes (res) dhe funksionin tjetër middleware në ciklin kërkesë-përgjigje (next).

- **Middleware të nivelit të aplikacionit:** Lidhën direkt me objektin app për çdo kërkesë ose rrugë specifike.
- **Middleware të palëve të treta:** Paketimet e NPM-së si cors (për lejim kërkesash ndër-origjinë), morgan (për regjistrimin e kërkesave në console), ose helmet (për siguri).
- **Menaxhimi i Gabimeve:** Funksione të posaçme middleware që marrin katër parametra (err, req, res, next) për të kapur dhe procesuar gabimet globalisht.

```
// Shembull i një middleware për logim app.use((req, res, next) => {
  console.log(`Kërkesë e re: ${req.method} në ${req.url}`); next(); // Kalon kontrollin te
  middleware apo router-i tjetër });
```

6. Përdorimet Kryesore

Express.js përdoret gjerësisht në raste të tilla si:

- **RESTful APIs:** Ndërtimi i backend-it për aplikacione single-page (SPA) ose celularë.
- **Microservices:** Shërbime të vogla e të pavarura që komunikojnë me njëra-tjetrën.
- **Aplikacione Web Tradicionale:** Renditje e faqeve me HTML dinamike në server.

7. Krahassimi me Korniza të Tjera

Express mbetet standardi i industrisë për shkak të thjeshtësisë dhe ekosistemit gjigand, por korniza të tjera moderne bazohen mbi të ose ofrojnë alternativa:

- **Koa.js:** Krijuar nga të njëjtët autorë të Express, me mbështetje më të mirë për async/await pa përdorur callback.
- **Fastify:** Një kornizë e fokusuar në performancë maksimalisht të lartë dhe kohë minimale ekzekutimi.
- **NestJS:** Një kornizë e strukturuar me TypeScript që përdor Express (ose Fastify) nën kapak.

